A Cost–Performance Comparison of Fine-Tuning Methods Across GPT-2 Scales and Tasks

Ángel Valencia
Complutense University of Madrid
angelvalenciabusiness@gmail.com

Abstract

Choosing a pretrained model size and a fine-tuning method under a compute budget is a joint decision, yet evidence on how the two interact is scattered across studies that differ in model, task and tuning effort. We study this interaction in GPT-2 (small, medium, large), comparing last-linear-layer training (LLL), LoRA and full fine-tuning on sentiment classification (SST-5, CFIMDB), paraphrase detection and sonnet generation under a shared protocol, and measuring performance, trainable parameters, peak memory and time per epoch. LoRA outperforms LLL by 4–13 points on every classification task and size, and matches full fine-tuning within noise while training under 0.5% of the parameters, although this translates into much smaller savings in memory and time. The benefit of scale is task dependent: LoRA saturates at small scale on CFIMDB and at medium scale on SST and sonnet, while paraphrase still improves at large scale, with diminishing returns. Cost grows roughly in proportion to model size, which on a fixed GPU becomes a feasibility limit: full fine-tuning of GPT-2 large on CFIMDB did not fit on our 15 GB GPU even at micro-batch 1, while LoRA large ran on all four tasks. LoRA at small-to-medium scale is therefore the best cost–accuracy default in our setup, although the best configuration depends on the task and the compute budget.

1 Introduction

Fine-tuning pretrained language models is the standard way to adapt them to downstream tasks [1], but growing model sizes make compute the main constraint. Practitioners must choose both a model size that fits their hardware and a fine-tuning method that makes the best use of it, yet how these choices interact, and what each costs, is hard to read from a literature where results differ in model, task and tuning effort. We ask: for a single model family (GPT-2), how do model size and fine-tuning method jointly determine performance and cost across different tasks?

Differences between methods or sizes are hard to attribute when comparisons differ in model, task and tuning effort [2]. Parameter-efficient fine-tuning has been studied extensively, including across scales [3], but we do not propose a new method. Instead, we offer a careful, reproducible[1], small-scale study where we fix the model family and vary size, method and task, measuring performance and cost together, under a modest compute budget of a Kaggle 15GB T4 GPU.

We implement GPT-2 [4] and AdamW [5] from scratch, and load the pretrained small (125M), medium (356M) and large (774M) GPT-2 weights from Hugging Face [6]. We compare last-linear-layer training (LLL), LoRA [7] and full fine-tuning across model sizes, excluding full fine-tuning at large scale, which was infeasible on our GPU. We evaluate these methods on sentiment classification (SST-5 [8] and CFIMDB, a polarized subset of IMDB reviews) and paraphrase detection (Quora Question Pairs [9]). On sonnet generation (Shakespeare's sonnets), we compare only LoRA and full fine-tuning, since LLL's classification head does not apply. We evaluate on the development sets and report accuracy for classification and chrF for sonnet generation, as means and standard deviations over three seeds. We also measure and analyse trainable parameters, peak memory and

[1] Code, configurations and per-run results are available at: https://github.com/angelvalen/gpt2-cost-performance-finetuning

time per epoch, using standard efficiency techniques (FP16 mixed precision, KV-cache for generation, gradient accumulation) to keep cost measurements realistic.

LoRA outperforms LLL by 4–13 points on every classification task and size, and matches full fine-tuning within noise while training under 0.5% of the parameters. The gains from scale depend on the task: LoRA saturates at small scale on CFIMDB and at medium scale on SST and sonnet, whereas paraphrase still improves at large scale, though by less than at the previous step. The parameter saving brings much smaller savings in memory and time, and cost grows roughly in proportion to model size, which on a fixed GPU becomes a feasibility limit: full fine-tuning of GPT-2 large did not fit even at micro-batch 1 on CFIMDB, whereas LoRA large ran on all four tasks. In our setup, LoRA at small-to-medium scale is therefore the best cost–accuracy default, although the best configuration depends on the task and the compute budget. Our conclusions are limited by a single learning rate per mode, a fixed LoRA configuration, three seeds and dev-set results. Cost measurements are also specific to a single Kaggle T4 GPU, and exclude five configurations that required gradient accumulation to fit in memory.

This project follows Stanford's CS224N Default Final Project [10], from which we use the starter instructions, datasets and report structure.

2 Related Work

2.1 Pretrained Language Models

Generative pretraining of Transformers [11] followed by supervised fine-tuning was shown to transfer effectively across a range of language understanding tasks [1], and subsequent work found that downstream performance tends to improve as models and pretraining data grow [12]. GPT-2 [4] further showed that a large Transformer language model pretrained on diverse web text acquires capabilities that transfer to many tasks, even in a zero-shot setting. These results motivate adapting large pretrained models to downstream tasks rather than training task-specific models from scratch.

2.2 Parameter-Efficient Fine-Tuning

Fully fine-tuning a large pretrained model requires storing a full model copy per task. Parameter-efficient fine-tuning (PEFT) methods address this by updating only a small number of parameters, often well under 1% of the total, while keeping the rest of the model frozen. Representative approaches include adapter modules, prefix tuning, and prompt tuning [2]. Low-Rank Adaptation (LoRA) [7] is among the most widely used: it freezes the pretrained weights and injects trainable low-rank decomposition matrices into the Transformer layers (typically the attention projections). Because the learned updates can be merged into the original weights after training, LoRA adds no inference latency, and it frequently matches the quality of full fine-tuning at a fraction of the trainable parameters.

2.3 Fine-Tuning at Scale

Zhang et al. (2024) [3] study fine-tuning in the data-limited regime, on 1B–16B bilingual LLMs, evaluating machine translation and summarization, comparing full-model tuning with prompt tuning and LoRA while varying fine-tuning data size, model size, pretraining data size and the number of PEFT parameters. They find that fine-tuning performance follows a power-based multiplicative joint scaling law between fine-tuning data size and each of the other factors. Scaling the model size benefits fine-tuning more than scaling pretraining data, whereas increasing the number of PEFT parameters is generally ineffective. Finally, they report that the best fine-tuning method depends strongly on the task and the amount of fine-tuning data.

3 Approach

3.1 Model Architecture

We implement GPT-2 [4] from scratch in PyTorch and load the pretrained weights from Hugging Face [6]. Text is tokenized with GPT-2's byte-level Byte-Pair Encoding (BPE) tokenizer [4], which

Table 1: Backbone configurations. d is the hidden size, L the number of transformer blocks, H the number of attention heads.

Backbone	d	L	H	Params
GPT-2 small	768	12	12	125.0M
GPT-2 medium	1024	24	16	355.9M
GPT-2 large	1280	36	20	774M

maps it to indices into the model's embedding matrix. The model sums token embeddings and learned absolute positional embeddings, and passes the result through a stack of L transformer blocks. Each block is pre-normalized: a layer normalization, masked multi-head self-attention and a residual connection, followed by a second layer normalization, a multilayer perceptron (MLP) and another residual connection. The model's parameters and dimensions for the considered sizes are shown in Table 1.

Self-attention [11] is the core component of the transformer, so to keep the study controlled and transparent we implemented masked (causal) multi-head self-attention from scratch in vectorized form, computing all heads in parallel with batched matrix products.

As in GPT-2, we apply dropout with $p_{\text{drop}} = 0.1$ after the attention and MLP sublayers, before the residual connection, and to the sum of the token and positional embeddings.

3.2 Classification tasks

For classification (sentiment analysis and paraphrase detection), let $h \in \mathbb{R}^d$ be the final-layer hidden state of the last input token, where d is the hidden size. We apply a linear classification head

$$\hat{y} = \text{softmax}(Wh + b),$$

with $W \in \mathbb{R}^{c \times d}$ and $b \in \mathbb{R}^c$, where c is the number of classes: 5 for SST-5 and 2 for CFIMDB and Quora. We apply dropout ($p = 0.1$) to h before the head. LLL is not evaluated on sonnet generation, since it is defined as training a newly added head, whereas generation uses GPT-2's pretrained output layer, which reuses the token embedding matrix E to compute the logits ($z = Eh$).

For paraphrase detection we use prompted classification: each pair of questions (s_1, s_2) from the Quora dataset is formatted as the prompt

Is s_1 a paraphrase of s_2? Answer "yes" or "no":

and classified by the same head from the hidden state of the last token.

3.3 Sonnet Generation

We implemented two decoding strategies, one stochastic (top-p sampling) and one deterministic (beam search), to compare their effect on sonnet generation. Both use a key–value cache and decode one prompt at a time: given the first three lines of a held-out sonnet, the model generates up to 128 new tokens, stopping early at the end-of-sequence token. Appendix A describes both algorithms and discusses why beam search underperforms, leading us to adopt top-p sampling for our experiments.

3.4 Fine-tuning modes

We compare three ways of adapting the pretrained backbone, applied identically across tasks and sizes. In last-linear-layer (LLL) fine-tuning, all backbone parameters are frozen and only the classification head is trained. In full fine-tuning, all backbone parameters (and the head, for classification) are trained. In LoRA [7], the pretrained weights are frozen and each adapted weight matrix $W_0 \in \mathbb{R}^{d \times d}$ is augmented with a trainable low-rank update,

$$h = W_0 x + \frac{\alpha}{r} BAx, \qquad A \in \mathbb{R}^{r \times d},\ B \in \mathbb{R}^{d \times r},$$

with B initialized to zero so that training starts from the pretrained model. We adapt the query, key, value and output projections of every attention layer (not the MLPs) with $r = 8$, $\alpha = 16$ and dropout 0.05 on the adapter input, train no bias terms, and use the `peft` library. The classification head is trained jointly. Table 5 reports the resulting numbers of trainable parameters.

3.5 Objective

For classification tasks, we train to maximize data log-likelihood, by minimizing the cross-entropy loss:

$$\mathcal{L}(y, \hat{y}) = -\sum_{i=1}^{c} y_i \log(\hat{y}_i)$$

Where $\hat{y}_i$ represents the probability for class $i \in \{1, \ldots, c\}$ and y is the one-hot encoded true label. Sonnet generation uses the same loss under a different label space: the "classes" become the vocabulary, $c = |V|$, and the target is the next token given the preceding context.

3.6 Optimizer

We use the AdamW optimizer [5]. It maintains Adam's first- and second-moment estimates of the gradient and applies weight decay decoupled from the gradient-based update, which regularizes the weights independently of the adaptive step size. We implemented the efficient bias correction version with multiplicative decoupled weight decay, from scratch. Pseudocode is given in Appendix B.

3.7 Training Choices

We used an early-stopping rule, where stopping is decided by the model's performance over the validation set. We used a patience of 5 epochs for classification tasks, versus 10 epochs for sonnet generation. And, in order to keep our results consistent with common practices, we implemented mixed-precision training with FP16. We note that BF16 is a more common precision for this purpose, but we ran our experiments on Kaggle's T4 GPU which does not support BF16. In addition to this, we implemented gradient acummulation to fit large models on our modest GPU by accumulating a small micro-batch into the desired effective one.

4 Experiments

4.1 Data

Datasets for our experiments were provided by Stanford's CS224N Default Final Project [10].

SST-5. The Stanford Sentiment Treebank [8] contains 11,855 single sentences taken from movie reviews. These sentences were parsed with the Stanford parser, yielding 215,154 unique phrases from the resulting parse trees. Each phrase was annotated by 3 human judges and assigned one of five labels: negative, somewhat negative, neutral, somewhat positive, or positive. The data is split into train (8,544 examples), dev (1,101 examples) and test (2,210 examples).

CFIMDB Dataset: The CFIMDB dataset is composed of 2,434 movie reviews that are highly polar. Every review is labeled as either negative or positive. Data is split into train (1,701 examples), dev (245 examples) and test (488 examples).

Quora Dataset: The Quora dataset contains 400,000 question pairs. Each pair is labeled to indicate whether the two questions are paraphrases of each other. Stanford provides a subset of this dataset, originally split into train (141,506 examples), dev (20,215 examples) and test (40,431 examples). Although, to speed up comparisons and reduce the disproportionate size of the dataset relative to the other tasks, we sampled and used only one-tenth of the training and development examples.

Sonnet Dataset: The Sonnet dataset is made up of 155 sonnets written by Shakespeare. Each sonnet has 14 lines and follows the rhyme scheme ABAB CDCD EFEF GG, where lines with the same letter ID rhyme with one another. Data is split into train (131 examples), dev (12 examples) and test (12 examples).

4.2 Evaluation method

Test labels for SST, CFIMDB, paraphrase detection and sonnet generation are withheld by Stanford's CS224N course, so we do not report test accuracy and all results are reported on the official development sets. However, since our goal is to compare configurations under identical conditions rather than to claim state-of-the-art absolute performance, development-set evaluation is sufficient: every

Table 2: Micro-batch size × gradient-accumulation steps per configuration. The effective batch is fixed per task (32 for SST and paraphrase, 8 for CFIMDB, 4 for sonnet).

Mode	Size	SST	CFIMDB	Para.	Son.
LLL	small / medium / large	32×1	8×1	32×1	–
LoRA	small	32×1	8×1	32×1	4×1
	medium	32×1	8×1	16×2	4×1
	large	32×1	4×2	4×8	4×1
Full	small	32×1	8×1	32×1	4×1
	medium	32×1	4×2	8×4	4×1
	large	–	–	–	–

configuration is scored on the same data with the same protocol, so differences between configurations remain interpretable, although absolute numbers are not directly comparable to published test results.

As performance metric for classification tasks we report accuracy. For sonnet generation, 3-line held-out sonnets are provided for the model to complete, and we evaluate the chrF score [13] between the golden sonnets and the generated ones, after removing the held-out part to only evaluate generated tokens.

4.3 Experimental Details

We ran a grid over tasks, fine-tuning modes and model sizes: (SST-5, CFIMDB, Quora paraphrase, sonnets) × (LLL, LoRA, full fine-tuning) × (GPT-2 small, medium, large). Every configuration was repeated over the same three seeds, and we report means and standard deviations. Two combinations are excluded: LLL on sonnet generation, since it trains a newly added classification head and therefore does not apply to generation, and full fine-tuning of GPT-2 large, which is generally impractical at this scale and did not fit on our 15 GB GPU even at micro-batch 1 on CFIMDB. This leaves 29 configurations.

Other configurations also exceeded the GPU's memory at their effective batch size. For these five we reduced the micro-batch and used gradient accumulation to keep the effective batch size fixed per task (Table 2).

We established a fixed hyperparameter configuration, which we selected based on experience and some non-extensive comparisons between the expected best candidates, shown in Table 3. Particularly, the learning rate (LR) was adapted to each fine-tuning method, while fixed on the size and task axis.

We note that larger models sometimes require a lower LR and more regularization than smaller models, since they are more exposed to overfitting or catastrophic forgetting [12, 14]. Since we fixed LR across size on our baseline configuration, we include targeted one-seed sanity runs with lower learning rates and stronger regularization on medium and large models to prevent them from under-performing on the baseline configuration. These runs showed no consistent gains beyond noise, so we keep the baseline configuration as the center of the study (details in Appendix C).

Code, configurations, seeds and per-run results are available at `https://github.com/angelvalen/gpt2-cost-performance-finetuning`.

5 Results and Analysis

We report and analyse results for our grid of (task × fine-tuning method × model size) experiments. First we present each configuration's performance, measured as accuracy for classification tasks and chrF for sonnet generation. Afterward, to evaluate computational tradeoffs, we present comparisons regarding trainable parameters, memory allocation, and training speed.

Table 3: Main hyperparameter configuration.

Parameter	Scope	Value
Weight decay	all	0.01
Last Head Dropout	all	0.1
LoRA r, alpha, dropout	all	8, 16, 0.05
LoRA target modules, bias	all	q, k, v, attn_dense; none
Learning rate	mode	LLL 1e-3; LoRA 1e-4; Full 1e-5
Effective Batch size	task	SST 32, CFIMDB 8, Para. 32, Sonnet 4
Patience	task	Classification 5, Sonnet 10
Top-p Sampling	task	p 0.9, temperature 0.9
Beam Generation	task	beams 5, length-pen. 0.6

Table 4: Baseline results on the development sets: mean $\pm$ std. over 3 seeds (accuracy in % for SST, CFIMDB and paraphrase detection; chrF for sonnet generation with top-p sampling). In each column, the best mean and all configurations within one std. of it are in bold.

Mode (LR)	Size	SST	CFIMDB	Paraphrase	Sonnet
LLL (10^{-3})	small	47.4 ± 0.5	86.0 ± 2.5	71.0 ± 1.5	–
	medium	49.1 ± 0.6	86.3 ± 0.2	74.5 ± 0.3	–
	large	50.4 ± 0.3	90.6 ± 0.0	76.5 ± 0.2	–
LoRA (10^{-4})	small	51.6 ± 0.1	$\mathbf{97.7 \pm 0.6}$	84.4 ± 0.3	26.3 ± 0.5
	medium	$\mathbf{55.2 \pm 0.5}$	$\mathbf{97.6 \pm 0.4}$	86.3 ± 0.3	$\mathbf{26.9 \pm 0.2}$
	large	$\mathbf{54.9 \pm 0.5}$	$\mathbf{97.3 \pm 0.6}$	$\mathbf{87.2 \pm 0.1}$	$\mathbf{27.0 \pm 0.4}$
Full (10^{-5})	small	50.1 ± 0.9	$\mathbf{98.0 \pm 0.7}$	84.5 ± 0.1	26.3 ± 0.5
	medium	$\mathbf{54.8 \pm 1.1}$	$\mathbf{97.7 \pm 0.5}$	85.8 ± 0.4	$\mathbf{27.0 \pm 0.2}$
	large	–	–	–	–

5.1 Performance Comparison

Table 4 is the project's cornerstone and shows the performance of each (model size $\times$ fine-tuning method) configuration across tasks. Some key takeaways we can derive from these results are the following.

1. LLL $<$ LoRA $\approx$ Full. On the three classification tasks, LoRA improves clearly on LLL at every size: by +4.2 / +6.1 / +4.5 points on SST, +11.7 / +11.3 / +6.7 on CFIMDB and +13.4 / +11.8 / +10.7 on paraphrase (small / medium / large). On CFIMDB and paraphrase the gap shrinks with size, because LLL keeps improving with scale while LoRA saturates (see below). LoRA matches full fine-tuning within noise at the two sizes where both were run: the difference is at most 0.5 points on every task except SST at small scale, where LoRA is 1.5 points higher (full fine-tuning has a std of 0.9 there). With three seeds we read this as no measurable difference, not as equivalence. LLL can only re-read the frozen last-layer features, so it succeeds only if the task information is already linearly decodable there. LoRA's attention updates change what the last token aggregates from the context, which is what the classification head reads. That attention-only low-rank updates suffice to match full fine-tuning agrees with Hu et al. (2022) [7], which suggests adaptation here mostly re-routes information the pretrained model already computes, although we did not test this.

2. Scaling differs across fine-tuning modes. LLL improves with model size on SST (47.4 $\rightarrow$ 49.1 $\rightarrow$ 50.4), paraphrase (71.0 $\rightarrow$ 74.5 $\rightarrow$ 76.5) and CFIMDB (86.0 $\rightarrow$ 86.3 $\rightarrow$ 90.6), although the small-to-medium step on CFIMDB is within noise. LoRA gains from small to medium on SST (+3.6) and paraphrase (+1.9), only marginally on sonnet (+0.6 chrF, of the order of the seed std) and not on CFIMDB.[2] From medium to large, LoRA plateaus on SST, CFIMDB and sonnet (changes of -0.3, -0.3 and $+0.1$, within noise), but still gains 0.9 points on paraphrase. We speculate that

[2]We speculate that CFIMDB is saturated at 97–98%: configurations that differ clearly on other tasks (e.g. LoRA small and medium, 3.6 points apart on SST) all land in this interval. The dataset is also small (1.7k training and 245 dev examples), so a difference of 0.4 points is a single dev example.

Table 5: Computational cost per configuration: trainable parameters (absolute and as a share of the model), peak memory and training time per epoch. The LLL range reflects task-dependent classification heads (2 classes: CFIMDB, paraphrase; 5 classes: SST). † Run with the largest power-of-two micro-batch that fit plus gradient accumulation (same effective batch), which also affects time; see Table 2.

			Peak memory (GB)				Time per epoch (min)			
Mode	Size	Trainable params	SST	CFIMDB	Para.	Son.	SST	CFIMDB	Para.	Son.
LLL	small	1.5–3.8k ($< 0.004\,\%$)	0.78	1.19	1.73	–	0.68	1.33	1.38	–
	medium	2.1–5.1k ($< 0.002\,\%$)	2.03	2.73	3.60	–	2.08	3.79	3.80	–
	large	2.6–6.4k ($< 0.001\,\%$)	4.23	5.30	6.57	–	4.93	8.69	8.87	–
LoRA	small	0.59M (0.47 %)	1.84	4.49	9.01	1.68	1.02	1.95	2.93	1.02
	medium	1.57M (0.44 %)	5.13	11.83	12.89†	3.82	3.00	5.74	7.75†	1.45
	large	2.95M (0.38 %)	10.13	13.71†	9.78†	7.42	6.79	10.73†	29.10†	2.57
Full	small	125.0M (100 %)	3.01	5.72	10.59	2.95	1.29	2.10	3.75	0.40
	medium	355.9M (100 %)	8.24	11.52†	12.14†	6.91	3.96	5.86†	12.72†	0.79
	large	774M (100 %)	–	–	–	–	–	–	–	–

this is because paraphrase, with 28k question pairs used for training, is the least saturated task in our grid: scale continues to buy generalizable signal, whereas SST-5, CFIMDB and the sonnet corpus are effectively saturated at the sizes we test. Full fine-tuning follows the same small-to-medium pattern as LoRA (+4.7 on SST, +1.3 on paraphrase, +0.7 chrF on sonnet, no change on CFIMDB), but without a large model we cannot tell whether it plateaus too.

3. The scale at which performance saturates depends on the task. With LoRA or full fine-tuning, CFIMDB is already saturated at small scale (97–98% [2]); SST and sonnet stop improving at medium scale (LoRA medium and large and full fine-tuning medium are all within one std of the best); and paraphrase has not saturated at large scale, where LoRA large (87.2) is the unique best, with no other configuration within one std (LoRA medium 86.3, full fine-tuning medium 85.8). The best configuration is therefore task dependent.

5.2 Computational cost

Alongside performance, we measure for every configuration the trainable parameters, the peak GPU memory allocated during training (`torch.cuda.max_memory_allocated`) and the wall-clock time per training epoch, averaged over three seeds. All runs used a Tesla T4 15 GB GPU on Kaggle, so absolute values are specific to our setup and we compare ratios. Epoch time includes the evaluation run every epoch for early stopping (forward-only passes over the training and dev sets for classification; dev-set generation for sonnet).

One caveat conditions every comparison. Five configurations did not fit in memory at their effective batch size, so we used the largest power-of-two micro-batch that fit, with gradient accumulation keeping the effective batch fixed (cells marked † in Table 5; settings in Table 2). In these cells, peak memory is set by the GPU capacity and the power-of-two step, and time per epoch by the smaller micro-batch, so neither reflects the intrinsic cost of the configuration. We therefore base quantitative comparisons of memory and time on the remaining cells, which ran at the effective batch size and are mutually comparable within a task, and use the micro-batches of the dagger cells as a coarse measure of memory capacity.

We note that time per epoch does not capture convergence, since different configurations may early stop faster, but we did not measure this.

1. LoRA saves parameters far more than memory or time. LoRA trains 0.38–0.47% of the parameters of full fine-tuning, and LLL only 1.5–6.4k ($< 0.004\%$), two to three orders of magnitude fewer than LoRA, at a cost of 4–13 accuracy points on classification (Table 4). The memory saving is much smaller. On comparable cells, LoRA saves 1.2–1.6 GB over full fine-tuning at small scale and 3.1 GB at medium scale on SST and sonnet, of the order of the backbone's gradients and Adam states (12 bytes per parameter in fp32: 1.5 and 4.3 GB). This is 38–45% on SST and sonnet but

only 15–22% on CFIMDB and paraphrase at small scale, consistent with LoRA still storing the activations needed to backpropagate through the frozen backbone, which are larger on those tasks (LoRA small peaks at 1.8 GB on SST vs. 4.5 and 9.0 GB on CFIMDB and paraphrase). At medium scale on CFIMDB and paraphrase, LoRA fit at least twice the micro-batch of full fine-tuning (8 vs. 4 and 16 vs. 8), so at equal micro-batch it saves at least about 3 and 2 GB. LLL is the cheapest by far, using 3.9–6.1× less memory than full fine-tuning since it needs no backward pass through the backbone (SST, small: 0.78 GB vs. 1.84 GB for LoRA and 3.01 GB for full fine-tuning).

In time, LoRA takes 0.76–0.93 of the time of full fine-tuning on comparable classification cells (a 7–24% saving) and LLL 0.37–0.63. Counting the backward pass as twice a forward pass and the per-epoch evaluation as one forward pass over the training set, an epoch costs about 4 forward-equivalents for full fine-tuning, 3 for LoRA (no weight gradients for frozen weights) and 2 for LLL (no backward through the backbone), i.e. ratios of 0.75 and 0.5, close to the 0.76–0.79 and 0.53 observed on SST and to the 0.78 for LoRA on paraphrase, although the other comparable cells deviate, in both directions (LoRA 0.93 and LLL 0.63 on CFIMDB; LLL 0.37 on paraphrase), which we did not investigate further. LoRA's time saving is thus modest relative to its parameter saving. Sonnet is the exception: LoRA epochs are 1.8–2.6× slower than full fine-tuning (1.45 vs. 0.79 min at medium, 1.02 vs. 0.40 at small), although all its cells share a micro-batch. With only 131 training sonnets, the epoch is probably dominated by the dev-set generation used for early stopping, which would hide LoRA's cheaper backward pass, but we did not time generation separately and cannot explain why LoRA is slower there.

2. Model size is the main driver of cost. On comparable classification cells, time per epoch grows roughly in proportion to the backbone: 2.8–3.1× from small to medium (backbone parameters: 2.85×) and 2.3–2.4× from medium to large (2.2×), while peak memory grows 2.1–2.8× and 1.8–2.1×, respectively. On sonnet, time grows only 1.4–2.0× per step although memory grows as elsewhere (1.9–2.3×), consistent with dev-set generation dominating the epoch. On the classification cells, a size step therefore changes cost by at least as much as the choice between LoRA and full fine-tuning (1.1–1.3× in time, 1.2–1.6× in memory), and by as much as the choice between LLL and full fine-tuning in time (1.6–2.7×); scaling from small to large costs 6.4–7.3× in time under LLL. The exception is LLL's memory, 3.9–6.1× below full fine-tuning, comparable to the 3.8–5.4× of scaling LLL from small to large.

3. Memory limits show up as accumulation. On a 15 GB GPU, memory is a feasibility constraint that gradient accumulation converts into time. Since each reduced micro-batch is the largest power of two that fit, Table 2 gives a rough indication of which configurations need more memory. At medium scale, LoRA fit at least twice the micro-batch of full fine-tuning on CFIMDB and paraphrase; full fine-tuning of GPT-2 large did not fit even at micro-batch 1 on CFIMDB, whereas LoRA large ran on all four tasks, with micro-batch 4 on CFIMDB and paraphrase. The time penalty can be large: LoRA large on paraphrase takes 29.1 min per epoch, 3.8× LoRA medium, versus about 2.3× for a size step on comparable cells, although we cannot separate the accumulation penalty from the size effect.

4. What the extra cost buys. LoRA on GPT-2 small outperforms LLL on GPT-2 large on all three classification tasks (Table 4) with 4.8×, 4.5× and 3.0× less time per epoch (SST, CFIMDB, paraphrase), and less memory on SST (1.8 vs. 4.2 GB) and CFIMDB (4.5 vs. 5.3 GB), but more on paraphrase (9.0 vs. 6.6 GB). Beyond medium scale, LoRA gains little for considerably more compute. From medium to large, SST (-0.3 points), CFIMDB (-0.3) and sonnet ($+0.1$ chrF) do not improve, at 2.3× the time and 2.0× the memory on SST and 1.8× and 1.9× on sonnet; on CFIMDB, LoRA large needed a smaller micro-batch to fit (8→4), indicating a larger memory footprint, although the cost is not directly comparable. Only paraphrase still gains ($+0.9$, vs. $+1.9$ at the previous step), at the price of a much smaller micro-batch (16→4). Compared with full fine-tuning, LoRA's savings come at no measurable quality cost: LoRA medium matches it on SST with 1.3× less time and 1.6× less memory, and on sonnet chrF is the same at both sizes (26.3 vs. 26.3, 26.9 vs. 27.0) with 43–45% less memory, although LoRA is slower there (the differences are within the seed standard deviation of 0.2–0.5, and chrF may be insensitive to the effect). Under this protocol (one LR per mode), LoRA at small to medium scale is therefore the best cost–accuracy default, with LLL as the cheapest option, at the price of 4–13 points on classification.

6 Conclusion

We studied how model size and fine-tuning method jointly determine performance and cost in GPT-2, comparing last-linear-layer training (LLL), LoRA and full fine-tuning at three scales on sentiment classification (SST-5, CFIMDB), paraphrase detection and sonnet generation.

Our findings follow three axes. On the choice of method, LoRA outperforms LLL by 4–13 points on every classification task and size, and matches full fine-tuning (which at large scale was infeasible on our GPU) within noise while training under 0.5% of the parameters. On the choice of size, the gains from scale are task dependent: LoRA saturates at small scale on CFIMDB and at medium scale on SST and sonnet, whereas paraphrase still improves at large scale, though by less than at the previous step. On cost, the parameter saving brings smaller savings in peak memory and time at equal micro-batch, and cost grows roughly in proportion to model size. On a 15 GB GPU this growth becomes a feasibility limit: full fine-tuning of GPT-2 large did not fit even at micro-batch 1 on CFIMDB, whereas LoRA large ran on all four tasks.

Taken together, accuracy saturates at small or medium scale on three of the four tasks while cost keeps growing, so LoRA at small-to-medium scale is the best cost–accuracy default in our setup. LLL, whose memory is 4–6$\times$ below full fine-tuning, is the cheapest option, at the price of 4–13 points on classification. The best choice ultimately depends on the task and the hardware.

Our conclusions are limited to one learning rate per mode, a fixed LoRA configuration, three seeds, dev-set results, no full fine-tuning on GPT-2 large; and an individual Kaggle T4 GPU, where five configurations required gradient accumulation and are excluded from our cost comparisons. Future work should vary the LoRA rank and adapted modules, tune the learning rate per size and task, and benchmark cost at a common micro-batch.

References

[1] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018.

[2] Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*, 2023.

[3] Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. When scaling meets LLM finetuning: The effect of data, model and finetuning method. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[6] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[8] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.

[9] Shankar Iyer, Nikhil Dandekar, and Kornel Csernai. First quora dataset release: Question pairs. `https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs`, 2017.

[10] Stanford University. CS 224N: Natural language processing with deep learning, default final project handout. `https://web.stanford.edu/class/cs224n/index.html#schedule`, 2026.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

[12] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[13] Maja Popović. chrF: Character n-gram f-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, 2015.

[14] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[15] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.

A Decoding Method

Top-p sampling At each step, the logits z are divided by a temperature T and normalized with a softmax, Top-p (nucleus) sampling [15] then sorts the tokens by probability, keeps the smallest set of most likely tokens whose cumulative probability reaches p (always including the most likely one), renormalizes their probabilities and samples from them. We use $p = 0.9$ and $T = 0.9$, which worked best in preliminary comparisons.

Beam search We maintain $B = 5$ partial sequences (beams). At each step, every unfinished beam is expanded with its B most probable next tokens, and the B best of the resulting candidates are kept, ranked by the length-normalized score S/ℓ^{α}, where S is the cumulative log-probability of the generated tokens, ℓ is the sequence length (counting prompt and added tokens) and $\alpha = 0.6$. Beams that emit the end-of-sequence token are finished and kept unchanged. The search stops when all beams are finished or after 128 tokens, and returns the beam with the highest normalized score.

Table 6: Sonnet generation (dev chrF, higher is better): top-p sampling vs. beam search, mean $\pm$ std over 3 seeds.

Mode	Size	Top-p	Beam search	Δ
LoRA	small	26.3 ± 0.5	13.6 ± 0.3	$+12.7$
	medium	26.9 ± 0.2	20.7 ± 1.3	$+6.2$
	large	27.0 ± 0.4	22.6 ± 1.6	$+4.4$
Full	small	26.3 ± 0.5	22.0 ± 0.3	$+4.3$
	medium	27.0 ± 0.2	23.6 ± 0.7	$+3.4$

Results We choose Top-p sampling as our primary generation technique for evaluation on this task, over the more deterministic approach with beam search generation, since the latter consistently performed worse on all configurations, as shown in Table 6.

In particular, we suffered from a well-studied [15] behavior of deterministic decoding methods such as beam search, where they can produce degenerate text consisting of repetitive loops. E.g.

And do I not love thee when I know not how,
And do I not think on life when I know not how,
And do I not love ourselves when we know not how,
...

B AdamW pseudocode

Algorithm 1 AdamW as implemented in this work.

Require: Learning rate η, decay rates $\beta_1, \beta_2 \in [0, 1)$, $\epsilon > 0$, weight decay λ, initial parameters θ_0

$m_0 \leftarrow 0,\ v_0 \leftarrow 0$
for $t = 1, 2, \ldots$ **do**
  $g_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$ ▷ minibatch gradient
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$ ▷ first moment
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$ ▷ second moment
  if bias correction enabled **then**
    $\eta_t \leftarrow \eta \sqrt{1 - \beta_2^t} \,/\, (1 - \beta_1^t)$ ▷ efficient bias correction
  else
    $\eta_t \leftarrow \eta$
  end if
  $\theta_t \leftarrow \theta_{t-1} - \eta_t\, m_t \,/\, (\sqrt{v_t} + \epsilon)$ ▷ gradient-based update
  $\theta_t \leftarrow (1 - \eta\lambda)\, \theta_t$ ▷ decoupled weight decay
end for
return θ_t

C Sanity Runs

Table 7 shows local (one seed) sanity runs, presenting hyperparameter changes from the baseline configuration, with their resulting performance. We lowered LR to configurations where increasing model size reported small or no improvement at all. In addition, we applied stronger regularization (dropout on the LLL head, LoRA dropout, weight decay) to medium size full fine-tuning. on CFIMDB, where fast convergence and the small dataset size motivated an overfitting diagnosis.

Table 7: Baseline 3-seed mean vs. learning-rate and regularization sensitivity runs on a single seed. Reg. denotes added regularization. LR changes are multiplicative factors on the base LR.

Task	Mode	Size	Change	Baseline	Tuned	Δ
SST	Full	small	LR $\times 3$	50.1 ± 0.9	50.8	$+0.7$
	Full	medium	LR $\div 2$	54.8 ± 1.1	55.0	$+0.2$
	LLL	large	LR $\div 3$	50.4 ± 0.3	50.0	-0.3
	LoRA	large	LR $\div 3$	54.9 ± 0.5	54.9	-0.1
CFIMDB	LLL	medium	LR $\div 2$	86.3 ± 0.2	87.3	$+1.1$
	LoRA	medium	LR $\div 2$	97.6 ± 0.4	97.6	0.0
	Full	medium	LR $\div 2$	97.7 ± 0.5	96.3	-1.4
	LLL	large	LR $\div 3$	90.6 ± 0.0	88.6	-2.0
	LoRA	large	LR $\div 3$	97.3 ± 0.6	97.1	-0.1
	LLL	medium	Reg.	86.3 ± 0.2	86.1	-0.1
	LoRA	medium	Reg.	97.6 ± 0.4	97.6	0.0
	Full	medium	Reg.	97.7 ± 0.5	98.0	$+0.3$
Paraphrase	Full	medium	LR $\div 2$	85.8 ± 0.4	86.0	$+0.2$
	LLL	large	LR $\div 3$	76.5 ± 0.2	76.5	0.0
Sonnet (top-p)	Full	medium	LR $\div 2$	27.0 ± 0.2	27.0	0.0
	LoRA	large	LR $\div 3$	27.0 ± 0.4	25.4	-1.6

Two patterns are informative. First, lowering the LR has no consistent direction: effects change sign across modes and sizes (e.g. CFIMDB LLL: $+1.1$ at medium, -2.0 at large), as expected if differences are noise (one CFIMDB dev example is 0.4 points). Second, extra regularization on medium CFIMDB leaves all three modes unchanged within noise, which is (weak, single-seed) evidence against overfitting and in favor of saturation as the explanation for flat CFIMDB scaling [2]. The one clear loss, LoRA large on sonnet with LR$\div 3$ (-1.6), is consistent with undertraining at a lower LR. This motivates us to maintain the baseline configuration as the center of our study, where the fixed hyperparameter configuration enhances comparability.